

SDSC Summer Institute 2026

Parallel Computing using MPI & Open MP

Igor Sfiligoi

Why do we need parallel programming?

- All modern hardware is parallel
 - Not even your phone does serial compute anymore!
- If you do not use parallel tools, you will be using only a fraction of hardware resources
 - I.e., it will take a long time to finish your compute task
- Hardware by itself cannot solve your problems
 - You need to split the problem in software
 - And explicitly deal with any conflicts and synchronization issues
(Remember the intro lecture?)

Anatomy of a modern CPU

- A modern CPU has between 4 and 64 physical compute cores
 - Each can be programmed independently
 - They are fully independent, compute wise
 - But compete for memory access
- Each physical compute core typically shows as 2 logical cores
 - Each can be programmed independently
 - They compete for physical compute resources (int and fp math, logic)
 - And, of course, memory access
- Each compute can execute vector instructions
 - Data parallelism, same operation on many data points
 - Vector width can vary ([with minimal penalty for larger vector sizes](#), up to ~512 bits)

Anatomy of a modern CPU

- A modern CPU has between 4 and 64 physical compute cores
 - Each can be programmed independently
 - They are fully independent, compute wise
 - But compete for memory access
- Each physical compute core type
 - Each can be programmed independently
 - They compete for physical resources
 - And, of course, memory access
- Each compute can execute vector operations
 - Data parallelism, same operation on many data points
 - Vector width can vary (**with minimal penalty for larger vector sizes**, up to ~512 bits)

Scalar compute is just a vector of width==1 (8, 16, 32 or 64 bits).
You are not using all CPU resources if each op is not 256 bits!

GPUs push it even further

- A modern GPU has $O(100)$ compute units (aka CUs, SMs)
- Each CU will handle $O(10)$ “warps”, similar to CPU logical cores
- Each “warp” operates on the equivalent of 32-wide (1kbit) CPU vector
- There are other differences, too
- But more on GPU programming another day

Anatomy of a modern compute node

- A laptop (or phone) likely has a single CPU
- Most high-end compute resources are multi-CPU
 - The Operating System (OS) hides this from you
 - You just see a list of all the available logical cores
- Why should you care if it has many CPUs?
 - Non-Uniform Memory Access (NUMA) considerations
 - It is much more expensive to access a memory location that is owned by a different physical CPU
- Physical CPU ID of your core can be obtained from the OS
 - But you must explicitly ask for that information (and act on it)

Anatomy of an HPC compute cluster

- A single compute node is great for lightweight compute
 - But can be limiting on the problem size you can solve
 - Both due to limited compute and memory available
- An HPC compute cluster “aggregates” many compute nodes
 - Unlike a multi-CPU node, you do not get “just more cores”
 - Each compute node is independent, memory is partitioned
 - But you do get fast, and relatively low-latency communication between them
- HPC programming is (conceptually) much harder
 - You must explicitly partition your problem, explicitly exchange information
 - But it avoids many traps inherent to shared-memory parallelism

Enter MPI and OpenMP

- You likely want both, as they address complementary needs
- **OpenMP – In-process parallelization (aka multi-threading)**
 - Allows you to use multiple compute cores
 - While having a common view of the data
 - Great when multiple threads reuse the same data
 - But be careful when writing shared data (i.e. conflicts/synchronization)
- **MPI – Multi-process coordination**
 - Allows you to use multiple nodes
 - No shared memory, so explicit memory partitioning/replication needed
 - Explicit information exchange, using library-handled messages

Enter MPI and OpenMP

- You likely want both, as they address complementary needs
- **OpenMP – In-process parallelization (aka multi-threading)**
 - Allows you to use multiple CPU cores
 - While having a common memory space
 - Great when multiple threads need to access shared data
 - But be careful of race conditions
- **MPI – Multi-process parallelization**
 - Allows you to use multiple CPU cores
 - No shared memory, so explicit memory partitioning/replication needed
 - Explicit information exchange, using library-handled messages

For best results use
OpenMP inside a single CPU/NUMA.
Use MPI to stitch together multiple CPUs.

OpenMP

What is OpenMP?

- OpenMP conceptually stands for **Open Multi-Processing**
- In practice, it is a formal specification
 - Which is evolving over time
 - Currently at version 6.0, but most compilers only support up to 5.0
 - Newer features only really useful for edge cases (will not touch them in this presentation)
 - You can find the full specification at <https://www.openmp.org>

OpenMP loop

- Most programmers start with a OpenMP loop
 - Great for when you have independent a data-parallel problem

```
for (int i = 0; i < n; i++) {  
    z[i] = a * x[i] + y[i];  
}
```

As written,
it is serial.

OpenMP loop

- Most programmers start with a OpenMP loop
 - Great for when you have independent a data-parallel problem

Note:

A (good) compiler will
already vectorize this code.
(Single-core parallelism)

No OpenMP needed.

```
for (int i = 0; i < n; i++) {  
    z[i] = a * x[i] + y[i];  
}
```

OpenMP loop

- Most programmers start with an OpenMP loop
 - Great for when you have independent a data-parallel problem
- All you need to make it parallel, is add a **pragma**
 - Understood by the compiler
 - But not part of the programming language

```
#pragma omp parallel for  
for (int i = 0; i < n; i++) {  
    z[i] = a * x[i] + y[i];  
}
```

Compiler still does
vectorization, too.

Now we are
also using
all the cores.

OpenMP loop

- Most programmers start with a OpenMP loop
 - Great for when you have independent a data-parallel problem
- All you need to make it parallel, is add a **pragma**
 - Understood by the compiler
 - But not part of the programming language

Most examples in this presentation are in C.
Very similar syntax in Fortran

```
!$omp parallel do  
do i=1,n  
    z(i) = a * x(i) + y(i);  
enddo
```

OpenMP loop

```
#pragma omp parallel for
for (int i = 0; i < 48; i++) {
    z[i] = a * x[i] + y[i];
}
```

Completed in 4 steps!
(Idealized)

CPU core 1



0	12	24	36
1	13	25	37
2	14	26	38
3	15	27	39

CPU core 2



4	16	28	40
5	17	29	41
6	18	30	42
7	19	31	43

CPU core 3



8	20	32	44
9	21	33	45
10	22	34	46
11	23	35	47

OpenMP loop

You can ensure
this splitting.

```
#pragma omp parallel for schedule(static,4)
for (int i = 0; i < 48; i++) {
    z[i] = a * x[i] + y[i];
}
```

Completed in 4 steps!
(Idealized)

CPU core 1

0	12	24	36
1	13	25	37
2	14	26	38
3	15	27	39

CPU core 2

4	16	28	40
5	17	29	41
6	18	30	42
7	19	31	43

CPU core 3

8	20	32	44
9	21	33	45
10	22	34	46
11	23	35	47

OpenMP loop

Alternative
(often default)

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < 48; i++) {
    z[i] = a * x[i] + y[i];
}
```

Completed in 4 steps!
(Idealized)

CPU core 1

0	4	8	12
1	5	9	13
2	6	10	14
3	7	11	15

CPU core 2

16	20	24	28
17	21	25	29
18	22	26	30
19	23	27	31

CPU core 3

32	36	40	44
33	37	41	45
34	38	42	46
35	39	43	47

OpenMP spec:
If chunk_size is not specified, the logical iteration space is divided into chunks that are approximately equal in size, and at most one chunk is distributed to each thread.

OpenMP unbalanced loop

- Not all loops are perfectly balanced
(Indeed, unfortunately, most non-trivial ones are not)
- When the imbalance is high, you may end up with unused cores

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    z[i] = a * x[i] + y[i];
    if ((i%12)<8) {
        w[(i/12)*8+ i%12] = b * x[i] + y[i];
    }
}
```

OpenMP unbalanced loop

```
#pragma omp parallel for schedule(static,4)
```

```
for (int i = 0; i < 36; i++) {  
    z[i] = a * x[i] + y[i];  
    if ((i%12)<8) {  
        w[(i/12)*8+ i%12] = b * x[i] + y[i];  
    }  
}
```

Completed in 6 steps.

Core 3 unused half of the time.

Work partitioned
at loop start.

Core 1

0	12	24
1	13	25
2	14	26
3	15	27

Core 2

4	16	28
5	17	29
6	18	30
7	19	31

Core 3

8	20	32
9	21	33
10	22	34
11	23	35

OpenMP unbalanced loop

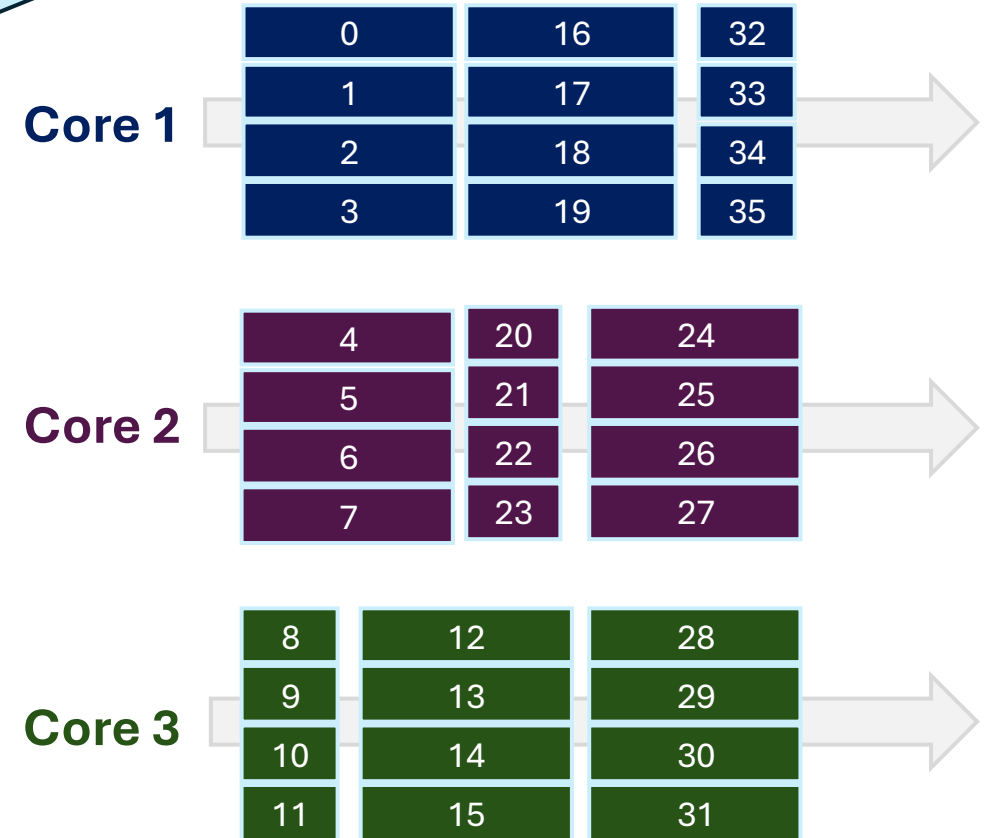
```
#pragma omp parallel for schedule(dynamic,4)
```

```
for (int i = 0; i < 36; i++) {  
    z[i] = a * x[i] + y[i];  
    if ((i%12)<8) {  
        w[(i/12)*8+ i%12] = b * x[i] + y[i];  
    }  
}
```

Completed in 5 steps.
(Idealized)

Next element
picked at runtime.

Warning:
Dynamic scheduling does
come with higher overhead



OpenMP loop with accumulated result

- Not all compute is completely independent
(Indeed, unfortunately, a very small subset is)
- Combining data is quite common

```
weighted_sum=0.0;

for (int i = 0; i < n; i++) {
    weighted_sum += x[i]*y[i];
}
```

OpenMP loop with accumulated result

- Not all compute is completely independent
(Indeed, unfortunately, a very small subset is)
- Combining data is quite common

Naïve parallelization
would result in
undefined behavior.

```
weighted_sum=0.0;

for (int i = 0; i < n; i++) {
    weighted_sum += x[i]*y[i];
}
```

OpenMP loop with accumulated result

- Not all compute is completely independent
(Indeed, unfortunately, a very small subset is)
- Combining data is quite common

OpenMP will follow your instructions blindly!
It is **your responsibility** to verify if it is legit!

Naïve parallelization
would result in
undefined behavior.

```
weighted_sum=0.0;  
#pragma omp parallel for  
for (int i = 0; i < n; i++){  
    weighted_sum += x[i]*y[i];  
}
```


OpenMP loop with accumulated result

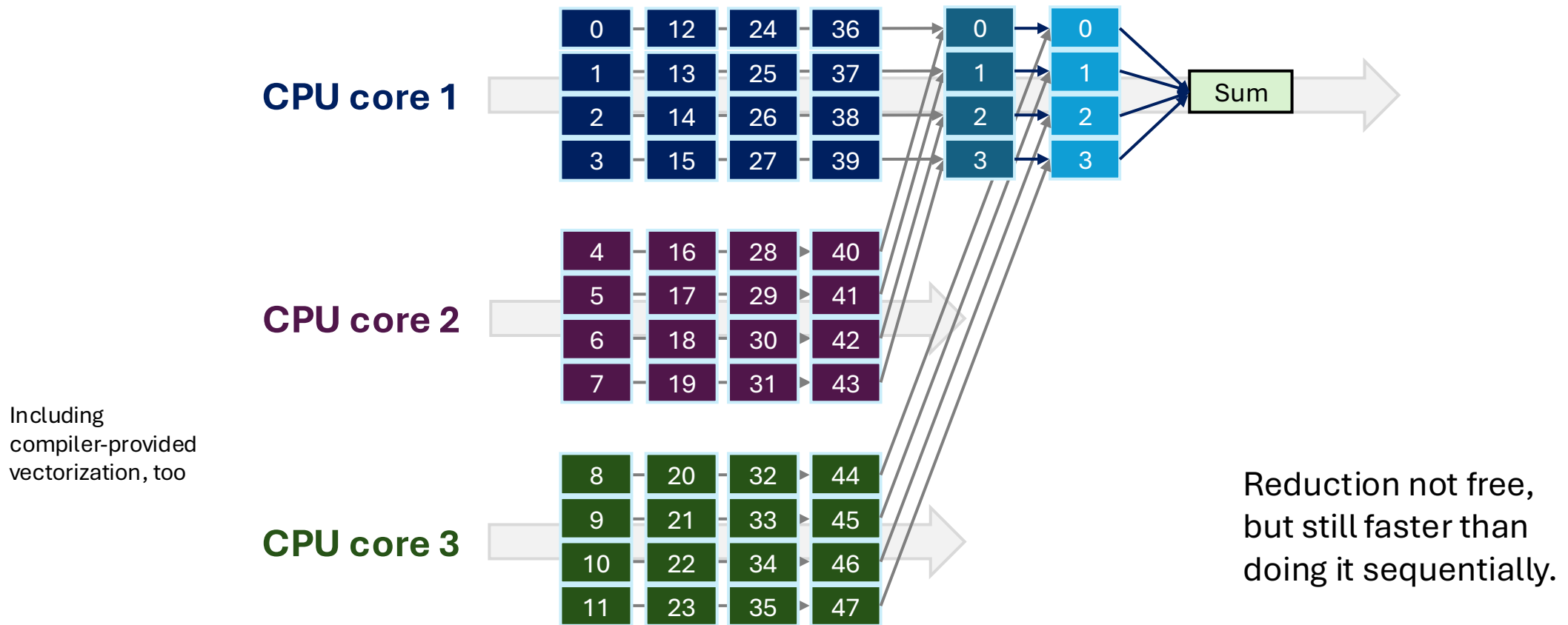
- Not all compute is completely independent
(Indeed, unfortunately, a very small subset is)
- Combining data is quite common

OpenMP can do some operations for you
(+, -, and, or, min, max)

Each thread will compute local subset,
final merge automatically happens just once.

```
weighted_sum=0.0;  
#pragma omp parallel for reduction(+:weighted_sum)  
for (int i = 0; i < n; i++) {  
    weighted_sum += x[i]*y[i];  
}
```

OpenMP loop with accumulated result



Mix and match OK

- Reductions and data-parallel writes not mutually exclusive
- Mixing and matching perfectly fine

```
wsum=0.0;
foundeq = false;
#pragma omp parallel for reduction(+:wsum) reduction(|:foundeq)
for (int i = 0; i < n; i++) {
    z[i] = a * x[i] + y[i];
    wsum += x[i]*y[i];
    foundeq |= ((b * x[i] + y[i]) == (x[i] + b*y[i]));
}
```

OpenMP atomic updates

- Reductions are great for regular operations
- But for rare events, using atomics may be more efficient

- OpenMP will ensure that two loops do not write at the same time

- Note:

Each atomic operation is expensive!

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    z[i] = a * x[i] + y[i];
    if ((x[i] + b * y[i]) == 0.0) {
        #pragma omp atomic update
        e7[i % 7] += z[i];
    }
}
```

Handling large scratch spaces

- Many problems need some scratch space
- Thread-local variables are trivial to use
 - Lifetime local to the loop, like their visibility
- You can also create local buffers
 - As long as they are small
 - Stack is a limited resource

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    float r[4*9];
    sgemm(4,5,9,a,x+i*4*5,y+i*5*9,r);
    for (j=0; j<4*9; j++)
        z[i*4*9+j] += r[j];
}
```

Handling large scratch spaces

- Many problems need some scratch space
- Thread-local variables are trivial to use
 - Lifetime local to the loop, like their visibility
- You can also create local buffers
 - As long as they are small
 - Stack is a limited resource

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    float r[4*9];
    sgemm(4,5,9,a,x+i*4*5,y+i*5*9,r);
    for (j=0; j<4*9; j++)
        z[i*4*9+j] += r[j];
}
```

Compiler will
vectorize at
this level.

Handling large scratch spaces

- Many problems need some scratch space
- Thread-local variables are trivial to use (in C/C++)
 - Lifetime local to the loop, like their visibility
- You can also create local buffers
 - As long as they are small
 - Stack is a limited resource

No loop-private variables in Fortran, but OpenMP allows to make them such

```
real, dimension(4,9) :: r
!$omp parallel do private(r)
do i=1,N
    call sgemm(a,x(:, :, i),y(:, :, i),r)
    z(:, :, i) = z(:, :, i) + r(:, :)
enddo
```

Handling large scratch spaces

- Many problems need some scratch space
- For large, and/or dynamic, scratch buffers, better to use a single global area
 - Partitioned logically per thread
 - Total number of threads and local thread ID easily obtainable

```
nthreads = omp_get_max_threads();
rglobal = new float[nthreads*K*M];
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    int my_omp = omp_get_num_threads();
    float *r = rglobal+my_omp*K*M;
    sgemm(K,L,M,a,x+i*K*L,y+i*L*M,r);
    for (j=0; j<K*M; j++)
        z[i*K*M+j] += r[j];
}
```


Handling large scratch spaces

- Many problems need some scratch space
- For large, and/or dynamic, scratch buffers, better to use a single global area
 - Partitioned logically per thread
 - Total number of threads and local thread ID easily obtainable

Typically
 $nthreads \ll n$

```
nthreads = omp_get_max_threads();  
rglobal = new float[nthreads*K*M];  
#pragma omp parallel for  
for (int i = 0; i < n; i++) {  
    int my_omp = omp_get_num_threads();  
    float *r = rglobal+my_omp*K*M;  
    sgemm(K,L,M,a,x+i*K*L,y+i*L*M,r);  
    for (j=0; j<K*M; j++)  
        z[i*K*M+j] += r[j];  
}
```

Handling large scratch spaces

- Many problems need some scratch space
- For large, and/or dynamic, scratch buffers, better to use a single global area
 - Partitioned logically per thread
 - Total number of threads and local thread ID easily obtainable

You can limit parallelism, if warranted

```
maxt = min(omp_get_max_threads(), 8);
rglobal = new float[maxt*K*M];
#pragma omp parallel for num_threads(maxt)
for (int i = 0; i < n; i++) {
    int my_omp = omp_get_num_threads();
    float *r = rglobal+my_omp*K*M;
    sgemm(K,L,M,a,x+i*K*L,y+i*L*M,r);
    for (j=0; j<K*M; j++)
        z[i*K*M+j] += r[j];
}
```

OpenMP nested loops

- Not all buffers are linear
 - Matrices and Tensors are quite common in real life

```
// z is 3D, x and y are 2D
```

```
for (int k = 0; k < P ; k++) {  
    for (int j = 0; j < M; j++) {  
        for (int i = 0; i < N; i++) {  
            z[(k*M+j)*N+i] = a * x[j*N+i] + b *y[k*M+j];  
        }  
    }  
}
```

OpenMP nested loops

- Not all buffers are linear
 - Matrices and Tensors are quite common in real life

! z is 3D, x and y are 2D

```
do k=1,P
  do j=1,M
    do i=1,N
      z(i,j,k) = a * x(i,j) + b * y(j,k);
    enddo
  enddo
enddo
```

Fortran equivalent
Easier to read

OpenMP nested loops

- Not all buffers are linear
 - Matrices and Tensors are quite common in real life
- You can parallelize as usual

```
// z is 3D, x and y are 2D
#pragma omp parallel for
for (int k = 0; k < P ; k++) {
    for (int j = 0; j < M; j++) {
        for (int i = 0; i < N; i++) {
            z[(k*M+j)*N+i] = a * x[j*N+i] + b *y[k*M+j];
        }
    }
}
```

OpenMP nested loops

- Not all buffers are linear
 - Matrices and Tensors are quite common in real life
- You can parallelize as usual

But what is P is small?
E.g. <4 ?

```
// z is 3D, x and y are 2D
#pragma omp parallel for
for (int k = 0; k < P ; k++) {
    for (int j = 0; j < M; j++) {
        for (int i = 0; i < N; i++) {
            z[(k*M+j)*N+i] = a * x[j*N+i] + b *y[k*M+j];
        }
    }
}
```

OpenMP nested loops

- Not all buffers are linear
 - Matrices and Tensors are quite common in real life
- You can merge the loops to increase parallelization potential

Total parallelism is now
 $N * M * P$

```
// z is 3D, x and y are 2D
#pragma omp parallel for collapse(3)
for (int k = 0; k < P ; k++) {
    for (int j = 0; j < M; j++) {
        for (int i = 0; i < N; i++) {
            z[(k*M+j)*N+i] = a * x[j*N+i] + b * y[k*M+j];
        }
    }
}
```

OpenMP nested loops

- Not all buffers are linear
 - Matrices and Tensors are quite common in real life
- You can merge the loops to increase parallelization potential

Can be partial collapse, too (i.e. ≤ 3)

```
// z is 3D, x and y are 2D
#pragma omp parallel for collapse(3)
for (int k = 0; k < P ; k++) {
    for (int j = 0; j < M; j++) {
        for (int i = 0; i < N; i++) {
            z[(k*M+j)*N+i] = a * x[j*N+i] + b * y[k*M+j];
        }
    }
}
```


Compiling and running OpenMP programs

- Most compilers don't enable OpenMP by default
 - You must explicitly pass a “enable OpenMP flag”
 - Most popular compilers use **-fopenmp**
- A OpenMP compiled executable will use all available CPU cores
 - Auto-detected
 - If you want to cap the number of cores in use, set **export OMP_NUM_THREADS=<MAX CORES>**
- When using only a subset of cores, Linux will pick a subset for you
 - You can explicitly pick the ones you like with **taskset -c <core range> <your executable and arguments>**

Compiling and running OpenMP programs

- Most compilers don't enable OpenMP by default
 - You must explicitly pass a “enable OpenMP flag”
 - Most popular compilers use **-fopenmp**
- A OpenMP compiled executable
 - Auto-detected
 - If you want to cap the number of cores
 - export OMP_NUM_THREADS=<MAX>**
- When using only a subset of cores, Linux will pick a subset for you
 - You can explicitly pick the ones you like with **taskset -c <core range> <your executable and arguments>**

Taskset will also limit the parallelism.
Can be used on its own, without the
need for OMP_NUM_THREADS.

Exercise time

- Log into Expanse
- Checkout the git repository, if you did not do it already
- Go into
3.4_parallel_computing_mpi_openmp/exercises
- Do the 3 exercises:
 - 1_axpy
 - 2_pi
 - 3_coll
- The provided source code is sequential
 - It is your task to make it parallel with OpenMP

MPI (Message Passing Interface)

